TEMA: Impresión de reportes

Resultados de Aprendizaje:

- Que los estudiantes puedan realizar aplicaciones con impresión de reportes

DESARROLLO

Cuando desarrollamos aplicsiones siempre tenemos la necesidad imperiosa de realizar aplicasiones que impriman reporetes , para esta guia aprenderemos como realizar esto

Requerimientos:

Para la realizacion de esta guia usted debe de contar con los siguientes elementos

- Contar con el proyecto comercial generado en la guia anterios
- Contar con un servidor de base de datos mysql

Genreracion de repotes

1. Para podere imprimer repores deberemos de instalar el paquete ❯ **pip install weasyprint** ¸ que es el paquete que no permitiira realizar la imprecion de los daros

Instalacion de complemetos windwos

2. Si esta en windows deber de instalar el compemento In MSYS2
3. Una vez instalado debera de ejecutar el comando **pacman -S mingw-w64-ucrt-x86_64-pango**

Generacion de vista de imprecion de datos

4. Para la genercion de la vista lo agregaremos el archivo **catalogo/views/producto.py** y agregaremos el sigueinte codigo

```python
from weasyprint import HTML
from ..models.producto import Producto
from django.http import HttpResponse


def generar_pdf_view(request):
    # 1. Obtener datos o contexto
    productos = Producto.objects.all()
    contexto = {'productos': productos}
    # 2. Renderizar el HTML con los datos    "Renderizar": Unknown word.
    html_string = render_to_string('producto/reporte.html', contexto)
    # 3. Convertir HTML a PDF
    pdf = HTML(string=html_string).write_pdf()
    # 4. Enviar la respuesta HTTP indicando que es un PDF
    response = HttpResponse(pdf, content_type='application/pdf')
    # 'inline' para vista previa, 'attachment' para descargar
    response['Content-Disposition'] = 'inline; filename="documento.ico.pdf"'
    return response
```

5. Hoy deberemos de agergar la plantilla donde rendereizaremos los datos en el reporete para ello crearemos el directorio catalogo/templates/producto/reporte.html , con el contenido siguiente

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Reporte de productos</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
    }
    h1 {
      display: flex;
      justify-content: center;

      color: #2563eb; /* Tailwind's blue-600 */
    }
    p {
      margin-bottom: 8px;
    }
    hr {
      margin: 16px 0;
    }
  </style>
</head>
```

```html
<body>
  <h1 >Reporte de productos</h1>
  {% for producto in productos %}
    <p >Nombre: {{ producto.nombre }}</p>
    <p >Descripción: {{ producto.descripcion }}</p>
    <p >Precio de compra: {{ producto.precio_compra }}</p>
    <p >Precio de venta: {{ producto.precio_venta }}</p>
    <p >Stock: {{ producto.stock }}</p>
    <p >Categoría: {{ producto.categoria }}</p>
    <hr>
  {% endfor %}
</body>
</html>
```

Tenga en cuenta que el diseño de nuestro informe deba realizalo directamente en cada informe

6. Como sigueinte paso agergaremos a nuestro arhivos de url el llamado a dicha direccion , modificando el archivo **catalogo/urls.py** y agregando lo siguiente

```python
from .views.producto import lista_productos, delete_producto, add_producto, update_producto, generar_pdf_view
urlpatterns = [
    path("reporte_productos/", generar_pdf_view, name="reporte_producto"),
```

7. Para finalizar agregaremos un boton en la plantilla de lista de productos modificando el archivo **catalogo/plantillas/producto/lista.html**

```html
{% block contenido %}
  <h1 class="font-bold  text-balance">Lista de Categorias</h1>
<a href="{% url "add_producto" %}" class="p-2 bg-blue-600 my-4 inline-block text-white rounded
hover:bg-blue-700">Agregar Producto</a>

<a href="{% url "reporte_producto" %}" class="p-2 bg-blue-600 my-4 inline-block text-white rounded hover:bg-blue-700"
target="_blank">Generar reporte</a>
```

8. Para finalizar solo nos hace falta probar nuestra aplicasion

	**EJERCICIOS DE PRÁCTICA:**